

Define Problem Statement and perform Exploratory Data Analysis

- Delhivery, India's largest and fastest-growing fully integrated logistics provider, generates massive volumes of trip-level operational data through its data engineering pipelines. This raw data contains complex, multi-row trip records, segment-level routing information, distance and time metrics, and diverse categorical attributes related to transportation, routes, and logistics operations.
- To support Delhivery's mission of building an intelligent, high-quality, and efficient logistics operating system, the data must be converted into reliable, structured, and analysis-ready features. The challenge lies in cleaning, merging, interpreting, and engineering features from this fragmented dataset so that it can be used effectively for forecasting, optimization, and decision-making.
- This project focuses on:
 - Cleaning and sanitizing raw trip data, handling missing values, inconsistencies, and outliers.
 - Merging multi-row trip entries into unified trip-level records using appropriate aggregations.
 - Engineering meaningful features from timestamps, location names, and routing information.
 - Evaluating relationships between actual vs. OSRM travel times and distances.
 - Standardizing, normalizing, and encoding columns for model readiness.
 - Deriving insights on network efficiency, route performance, and operational bottlenecks.
- By transforming raw logistics data into high-quality analytical features, this work helps the Data Science team build robust forecasting models, optimize transportation routes, and drive better strategic and operational decisions—ultimately contributing to Delhivery's competitive advantage in speed, efficiency, and profitability.

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Importing Data

```
In [ ]: df = pd.read_csv("/content/drive/MyDrive/01. Data Set for Colab Notebooks/delhivery
```

```
In [ ]: df.head(3)
```

Out []:

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	so
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	INC
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	INC
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	INC

3 rows × 24 columns



Basic Data Exploration

```
In [ ]: print(f"The dataset has {df.shape[0]} rows and {df.shape[1]} columns")
```

The dataset has 144867 rows and 24 columns

```
In [ ]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   data                                  144867 non-null object
 1   trip_creation_time                   144867 non-null object
 2   route_schedule_uuid                 144867 non-null object
 3   route_type                           144867 non-null object
 4   trip_uuid                           144867 non-null object
 5   source_center                       144867 non-null object
 6   source_name                         144574 non-null object
 7   destination_center                  144867 non-null object
 8   destination_name                    144606 non-null object
 9   od_start_time                       144867 non-null object
10  od_end_time                         144867 non-null object
11  start_scan_to_end_scan               144867 non-null float64
12  is_cutoff                           144867 non-null bool
13  cutoff_factor                       144867 non-null int64
14  cutoff_timestamp                    144867 non-null object
15  actual_distance_to_destination       144867 non-null float64
16  actual_time                         144867 non-null float64
17  osrm_time                           144867 non-null float64
18  osrm_distance                       144867 non-null float64
19  factor                              144867 non-null float64
20  segment_actual_time                 144867 non-null float64
21  segment_osrm_time                   144867 non-null float64
22  segment_osrm_distance               144867 non-null float64
23  segment_factor                      144867 non-null float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB

```

- It describes the distribution of the numeric attributes non-graphically

In []: `df.describe()`

Out[]:

	start_scan_to_end_scan	cutoff_factor	actual_distance_to_destination	actual_time
count	144867.000000	144867.000000	144867.000000	144867.000000
mean	961.262986	232.926567	234.073372	416.927527
std	1037.012769	344.755577	344.990009	598.103621
min	20.000000	9.000000	9.000045	9.000000
25%	161.000000	22.000000	23.355874	51.000000
50%	449.000000	66.000000	66.126571	132.000000
75%	1634.000000	286.000000	286.708875	513.000000
max	7898.000000	1927.000000	1927.447705	4532.000000

- It describes the distribution of the string attributes non-graphically

```
In [ ]: df.describe(include="O")
```

Out []:

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid
count	144867	144867	144867	144867	144867
unique	2	14817	1504	2	14817
top	training	2018-09-22 04:55:04.835022	thanos::sroute:4029a8a2-6c74-4b7e-a6d8-f9e069f...	FTL	trip 15375921048347612
freq	104858	101	1812	99660	10

- Column wise - count of unique values

```
In [ ]: df.nunique()
```


Out[]:

	0
data	2
trip_creation_time	14817
route_schedule_uuid	1504
route_type	2
trip_uuid	14817
source_center	1508
source_name	1498
destination_center	1481
destination_name	1468
od_start_time	26369
od_end_time	26369
start_scan_to_end_scan	1915
is_cutoff	2
cutoff_factor	501
cutoff_timestamp	93180
actual_distance_to_destination	144515
actual_time	3182
osrm_time	1531
osrm_distance	138046
factor	45641
segment_actual_time	747
segment_osrm_time	214
segment_osrm_distance	113799
segment_factor	5675

dtype: int64

Missing Value Detection

In []:

```
df.isnull().sum()
```

Out[]:

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	293
destination_center	0
destination_name	261
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0

dtype: int64

- source_name has 293 null values
- destination_name has 261 null values
- But these missing values don't impact our analysis as we have source_center and destination_center

Introducing Feature 1 : unique key - to handle multi layer under each trip_uuid

```
In [ ]: df['unique_key'] = df['trip_uuid'] + df['source_center'] + df['destination_center']
```

```
In [ ]: df.head(3)
```

```
Out [ ]:
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	so
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	trip- INE
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	trip- INE
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	trip- INE

3 rows × 25 columns



```
In [ ]: cols = ['segment_actual_time', 'segment_osrm_distance', 'segment_osrm_time']
for c in cols:
    df[c + '_sum'] = df.groupby('unique_key')[c].cumsum()

df[[c + '_sum' for c in cols]]
```

Out[]:

	segment_actual_time_sum	segment_osrm_distance_sum	segment_osrm_time_sum
0	14.0	11.9653	11.0
1	24.0	21.7243	20.0
2	40.0	32.5395	27.0
3	61.0	45.5619	39.0
4	67.0	49.4772	44.0
...	...	...	...
144862	92.0	65.3487	94.0
144863	118.0	82.7212	115.0
144864	138.0	103.4265	149.0
144865	155.0	122.3150	176.0
144866	423.0	131.1238	185.0

144867 rows × 3 columns

- Aggregating at Sub-Journey Level

In []:

```
create_unique_dict = {
    'data' : 'first',
    'trip_creation_time' : 'first',
    'route_schedule_uuid' : 'first',
    'route_type' : 'first',
    'trip_uuid' : 'first',
    'source_center' : 'first',
    'source_name' : 'first',

    'destination_center' : 'last',
    'destination_name' : 'last',

    'od_start_time' : 'first',
    'od_end_time' : 'first',
    'start_scan_to_end_scan' : 'first',

    'actual_distance_to_destination' : 'last',
    'actual_time' : 'last',

    'osrm_time' : 'last',
    'osrm_distance' : 'last',

    'segment_actual_time_sum' : 'last',
    'segment_osrm_distance_sum' : 'last',
    'segment_osrm_time_sum' : 'last',
}
```

```
}
```

- Groupby mini-trips, sorting by time

```
In [ ]: unique = df.groupby('unique_key').agg(create_unique_dict).reset_index()  
unique = unique.sort_values(by=['unique_key', 'od_end_time'], ascending=True).reset_
```

```
In [ ]: unique.head()
```

Out []:

	index	unique_key	data	trip_creation_time	r
0	0	153671041653548748IND209304AAAIND000000ACB	trip- training	2018-09-12 00:00:16.535741	than
1	1	153671041653548748IND462022AAAIND209304AAA	trip- training	2018-09-12 00:00:16.535741	than
2	2	153671042288605164IND561203AABIND562101AAA	trip- training	2018-09-12 00:00:22.886430	than
3	3	153671042288605164IND572101AAAIND561203AAB	trip- training	2018-09-12 00:00:22.886430	than
4	4	153671043369099517IND000000ACBIND160002AAC	trip- training	2018-09-12 00:00:33.691250	than

5 rows × 21 columns



```
In [ ]: unique.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26368 entries, 0 to 26367
Data columns (total 21 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   index                                26368 non-null  int64
 1   unique_key                           26368 non-null  object
 2   data                                  26368 non-null  object
 3   trip_creation_time                   26368 non-null  object
 4   route_schedule_uuid                  26368 non-null  object
 5   route_type                           26368 non-null  object
 6   trip_uuid                            26368 non-null  object
 7   source_center                        26368 non-null  object
 8   source_name                          26302 non-null  object
 9   destination_center                   26368 non-null  object
10   destination_name                     26287 non-null  object
11   od_start_time                        26368 non-null  object
12   od_end_time                          26368 non-null  object
13   start_scan_to_end_scan               26368 non-null  float64
14   actual_distance_to_destination        26368 non-null  float64
15   actual_time                          26368 non-null  float64
16   osrm_time                            26368 non-null  float64
17   osrm_distance                        26368 non-null  float64
18   segment_actual_time_sum              26368 non-null  float64
19   segment_osrm_distance_sum            26368 non-null  float64
20   segment_osrm_time_sum                26368 non-null  float64
dtypes: float64(8), int64(1), object(12)
memory usage: 4.2+ MB

```

Introducing Feature 2 : od_time_diff_mins - Time taken between od_start_time and od_end_time

```

In [ ]: unique['od_start_time'] = pd.to_datetime(unique['od_start_time'])
        unique['od_end_time'] = pd.to_datetime(unique['od_end_time'])

        unique['od_time_diff_mins'] = (unique['od_end_time'] - unique['od_start_time']).dt.
        unique['od_time_diff_mins']

```

Out[]: **od_time_diff_mins**

0	1260.604421
1	999.505379
2	58.832388
3	122.779486
4	834.638929
...	...
26363	62.115193
26364	91.087797
26365	44.174403
26366	287.474007
26367	66.933565

26368 rows × 1 columns

dtype: float64

Aggregating at trip level

```
In [ ]: create_trip_dict = {

    'data' : 'first',
    'trip_creation_time': 'first',
    'route_schedule_uuid' : 'first',
    'route_type' : 'first',
    'trip_uuid' : 'first',

    'source_center' : 'first',
    'source_name' : 'first',

    'destination_center' : 'last',
    'destination_name' : 'last',

    'start_scan_to_end_scan' : 'sum',
    'od_time_diff_mins' : 'sum',

    'actual_distance_to_destination' : 'sum',
    'actual_time' : 'sum',
    'osrm_time' : 'sum',
    'osrm_distance' : 'sum',

    'segment_actual_time_sum' : 'sum',
    'segment_osrm_distance_sum' : 'sum',
    'segment_osrm_time_sum' : 'sum',
```

}

```
In [ ]: trip = unique.groupby('trip_uuid').agg(create_trip_dict).reset_index(drop = True)
```

```
In [ ]: trip.head()
```

```
Out [ ]:
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	sc
--	------	--------------------	---------------------	------------	-----------	----

0	training	2018-09-12 00:00:16.535741	thanos::sroute:d7c989ba- a29b-4a0b-b2f4- 288cdc6...	FTL	153671041653548748	trip- 153671041653548748	IN
1	training	2018-09-12 00:00:22.886430	thanos::sroute:3a1b0ab2- bb0b-4c53-8c59- eb2a2c0...	Carting	153671042288605164	trip- 153671042288605164	IN
2	training	2018-09-12 00:00:33.691250	thanos::sroute:de5e208e- 7641-45e6-8100- 4d9fb1e...	FTL	153671043369099517	trip- 153671043369099517	IN
3	training	2018-09-12 00:01:00.113710	thanos::sroute:f0176492- a679-4597-8332- bbd1c7f...	Carting	153671046011330457	trip- 153671046011330457	IN
4	training	2018-09-12 00:02:09.740725	thanos::sroute:d9f07b12- 65e0-4f3b-bec8- df06134...	FTL	153671052974046625	trip- 153671052974046625	IN

```
In [ ]: trip[trip['trip_uuid'] == 'trip-153741093647649320']
```

```
Out [ ]:
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid
--	------	--------------------	---------------------	------------	-----------

5919	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320
------	----------	-------------------------------	---	---------	--------------------

```
In [ ]: trip[['actual_time', 'segment_actual_time_sum']]
```


Out[]:

	actual_time	segment_actual_time_sum
0	1562.0	1548.0
1	143.0	141.0
2	3347.0	3308.0
3	59.0	59.0
4	341.0	340.0
...	...	...
14812	83.0	82.0
14813	21.0	21.0
14814	282.0	281.0
14815	264.0	258.0
14816	275.0	274.0

14817 rows × 2 columns

In []:

```
trip[['actual_distance_to_destination', 'osrm_distance']]
```

Out[]:

	actual_distance_to_destination	osrm_distance
0	824.732854	991.3523
1	73.186911	85.1110
2	1927.404273	2354.0665
3	17.175274	19.6800
4	127.448500	146.7918
...	...	...
14812	57.762332	73.4630
14813	15.513784	16.0882
14814	38.684839	58.9037
14815	134.723836	171.1103
14816	66.081533	80.5787

14817 rows × 2 columns

Detecting Missing Value in trip

```
In [ ]: missing_value = pd.DataFrame({'Missing Value': trip.isnull().sum(), 'Percentage': (tr
missing_value.sort_values(by='Percentage', ascending=False)
```

```
Out[ ]:
```

	Missing Value	Percentage
source_name	10	0.067490
destination_name	8	0.053992
trip_creation_time	0	0.000000
data	0	0.000000
route_schedule_uuid	0	0.000000
route_type	0	0.000000
source_center	0	0.000000
trip_uuid	0	0.000000
destination_center	0	0.000000
start_scan_to_end_scan	0	0.000000
od_time_diff_mins	0	0.000000
actual_distance_to_destination	0	0.000000
actual_time	0	0.000000
osrm_time	0	0.000000
osrm_distance	0	0.000000
segment_actual_time_sum	0	0.000000
segment_osrm_distance_sum	0	0.000000
segment_osrm_time_sum	0	0.000000

```
In [ ]: trip.shape
```

```
Out[ ]: (14817, 18)
```

Removing Missing value from trip data

```
In [ ]: trip = trip.dropna(subset=['source_name', 'destination_name'])
```

```
In [ ]: missing_value = pd.DataFrame({'Missing Vallue': trip.isnull().sum(), 'Percentage': (t
missing_value.sort_values(by='Percentage', ascending=False)
```

Out[]:

	Missing Vallue	Percentage
data	0	0.0
trip_creation_time	0	0.0
route_schedule_uuid	0	0.0
route_type	0	0.0
trip_uuid	0	0.0
source_center	0	0.0
source_name	0	0.0
destination_center	0	0.0
destination_name	0	0.0
start_scan_to_end_scan	0	0.0
od_time_diff_mins	0	0.0
actual_distance_to_destination	0	0.0
actual_time	0	0.0
osrm_time	0	0.0
osrm_distance	0	0.0
segment_actual_time_sum	0	0.0
segment_osrm_distance_sum	0	0.0
segment_osrm_time_sum	0	0.0

In []: `trip.shape`

Out[]: (14800, 18)

Lowering the case of source_name and destination_name

```
In [ ]: trip['destination_name'] = trip['destination_name'].str.lower()
trip['source_name'] = trip['source_name'].str.lower()
```

Defining Function to strip out State Name

```
In [ ]: def place2state(x):
    state = x.split('(')[1] #splitting state based on "("
    return state[:-1] #removing ')' from end
```

Defining Function to strip out City

```
In [ ]: def place2city(x):
        # removing state
        city = x.split(' ')[0]
        # picking the first city name
        city = city.split('_')[0]

        # Now dealing with edge cases
        # ['PNQ Pashan DPC', 'Bhopal MP Nagar', 'HBR Layout PC',
        # 'PNQ Rahatani DPC', 'Pune Balaji Nagar', 'Mumbai Antop Hill']

        if city == 'pnq vadgaon sheri dpc':
            return 'vadgaonsheri'
        elif city in ['pnq pashan dpc', 'pnq rahatani dpc', 'pune balaji nagar']:
            return 'pune'
        elif city == 'hbr layout pc':
            return 'bengaluru'
        elif city == 'bhopal mp nagar':
            return 'bhopal'
        elif city == 'mumbai antop hill':
            return 'mumbai'
        else:
            return city
```

Defining Function to strip out Place

```
In [ ]: def place2city_place(x):

        # We will remove state
        x = x.split(' ')[0]

        len1 = len(x.split('_'))

        if len1 >= 3:
            return x.split('_')[1]

        # Small cities have same city and place name
        if len1 == 2:
            return x.split('_')[0]

        return x.split(' ')[0]
```

Defining Function to strip out Place Code

```
In [ ]: def place2code(x):
        # Removing State
        x = x.split(' ')[0]

        if len(x.split('_')) >= 3 :
```

```

return x.split('_')[-1]

return 'none'

```

Introducing 8 new features

```

In [ ]: trip['source_state'] = trip['source_name'].apply(lambda x: place2state(x))
trip['source_city'] = trip['source_name'].apply(lambda x: place2city(x))
trip['source_place'] = trip['source_name'].apply(lambda x: place2city_place(x))
trip['source_code'] = trip['source_name'].apply(lambda x: place2code(x))

trip['destination_state'] = trip['destination_name'].apply(lambda x: place2state(x))
trip['destination_city'] = trip['destination_name'].apply(lambda x: place2city(x))
trip['destination_place'] = trip['destination_name'].apply(lambda x: place2city_place(x))
trip['destination_code'] = trip['destination_name'].apply(lambda x: place2code(x))

```

```

In [ ]: trip[['source_state', 'source_city', 'source_place', 'source_code', 'destination_state', 'destination_city', 'destination_place', 'destination_code']]

```

```

Out[ ]:

```

	source_state	source_city	source_place	source_code	destination_state	destination_city	destination_place
0	uttar pradesh	kanpur	central	6	uttar pradesh	kanpur	central
1	karnataka	doddablpur	chikadpp	d	karnataka	doddablpur	chikadpp
2	haryana	gurgaon	bilaspur	hb	haryana	gurgaon	bilaspur
3	maharashtra	mumbai hub	mumbai	none	maharashtra	mumbai	central
4	karnataka	bellary	bellary	none	karnataka	santhoshnagar	central
...	...	...	...	...	...	...	...
14812	punjab	chandigarh	mehmdpur	h	punjab	chandigarh	central
14813	haryana	fbid	balabhgarh	dpc	haryana	faridkot	central
14814	uttar pradesh	kanpur	govndngr	dc	uttar pradesh	kanpur	central
14815	tamil nadu	tirunelveli	vdckusrt	i	tamil nadu	tiruchchur	central
14816	karnataka	sandur	wrdn1dpp	d	karnataka	santhoshnagar	central

14800 rows × 8 columns



- This will help us to make insights on
 - Busiest route
 - popular route

Splitting the trip_creation_time to fetch insights

Introducing 6 more features

```
In [ ]: trip['trip_creation_time'] = pd.to_datetime(trip['trip_creation_time']) # Converti

trip['trip_year'] = trip['trip_creation_time'].dt.year
trip['trip_month'] = trip['trip_creation_time'].dt.month
trip['trip_hour'] = trip['trip_creation_time'].dt.hour
trip['trip_day'] = trip['trip_creation_time'].dt.day
trip['trip_week'] = trip['trip_creation_time'].dt.isocalendar().week
trip['trip_dayofweek'] = trip['trip_creation_time'].dt.dayofweek
```

- This will help us to get insights on :
 - Busiest Month
 - Busiest Week
 - Busiest Day

```
In [ ]: trip.head()
```

```
Out[ ]:
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	sc
0	training	2018-09-12 00:00:16.535741	thanos::sroute:d7c989ba- a29b-4a0b-b2f4- 288cdc6...	FTL	153671041653548748	IN
1	training	2018-09-12 00:00:22.886430	thanos::sroute:3a1b0ab2- bb0b-4c53-8c59- eb2a2c0...	Carting	153671042288605164	IN
2	training	2018-09-12 00:00:33.691250	thanos::sroute:de5e208e- 7641-45e6-8100- 4d9fb1e...	FTL	153671043369099517	IN
3	training	2018-09-12 00:01:00.113710	thanos::sroute:f0176492- a679-4597-8332- bbd1c7f...	Carting	153671046011330457	IN
4	training	2018-09-12 00:02:09.740725	thanos::sroute:d9f07b12- 65e0-4f3b-bec8- df06134...	FTL	153671052974046625	IN

5 rows × 32 columns



```
In [ ]: trip.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 14800 entries, 0 to 14816
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  14800 non-null  object
1   trip_creation_time                    14800 non-null  datetime64[ns]
2   route_schedule_uuid                  14800 non-null  object
3   route_type                           14800 non-null  object
4   trip_uuid                            14800 non-null  object
5   source_center                        14800 non-null  object
6   source_name                          14800 non-null  object
7   destination_center                   14800 non-null  object
8   destination_name                     14800 non-null  object
9   start_scan_to_end_scan               14800 non-null  float64
10  od_time_diff_mins                    14800 non-null  float64
11  actual_distance_to_destination        14800 non-null  float64
12  actual_time                          14800 non-null  float64
13  osrm_time                            14800 non-null  float64
14  osrm_distance                        14800 non-null  float64
15  segment_actual_time_sum               14800 non-null  float64
16  segment_osrm_distance_sum             14800 non-null  float64
17  segment_osrm_time_sum                 14800 non-null  float64
18  source_state                          14800 non-null  object
19  source_city                          14800 non-null  object
20  source_place                          14800 non-null  object
21  source_code                           14800 non-null  object
22  destination_state                     14800 non-null  object
23  destination_city                      14800 non-null  object
24  destination_place                     14800 non-null  object
25  destination_code                      14800 non-null  object
26  trip_year                            14800 non-null  int32
27  trip_month                           14800 non-null  int32
28  trip_hour                            14800 non-null  int32
29  trip_day                             14800 non-null  int32
30  trip_week                            14800 non-null  UInt32
31  trip_dayofweek                        14800 non-null  int32
dtypes: UInt32(1), datetime64[ns](1), float64(9), int32(5), object(16)
memory usage: 3.4+ MB

```

Outlier Detection

```

In [ ]: numeric_cols = ['start_scan_to_end_scan', 'actual_distance_to_destination', 'actual
                        'osrm_distance', 'segment_actual_time_sum', 'segment_osrm_distance_sum'
                        'segment_osrm_time_sum', 'od_time_diff_mins']

```

```

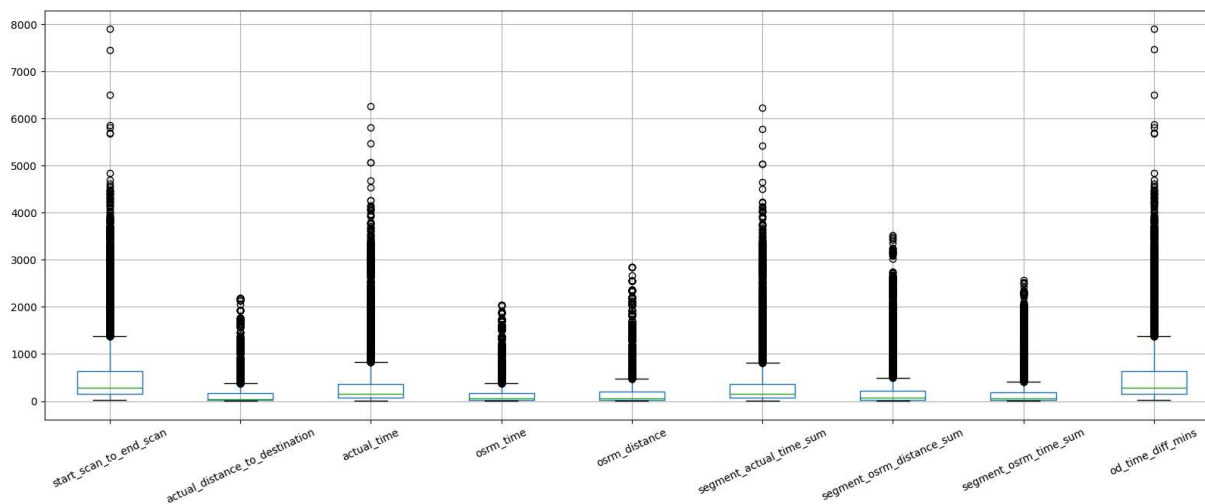
In [ ]: trip[numeric_cols].boxplot(rot=25, figsize=(20,7))

```

```

Out[ ]: <Axes: >

```



- All the numeric attributes are having outliers

Handling the Outliers - Using IQR

```
In [ ]: Q1 = trip[numeric_cols].quantile(0.25)
        Q3 = trip[numeric_cols].quantile(0.75)
```

```
In [ ]: IQR = Q3 - Q1
```

```
In [ ]: trip = trip[~((trip[numeric_cols] < (Q1 - 1.5 * IQR)) | (trip[numeric_cols] > (Q3 +
```

```
In [ ]: trip.head()
```

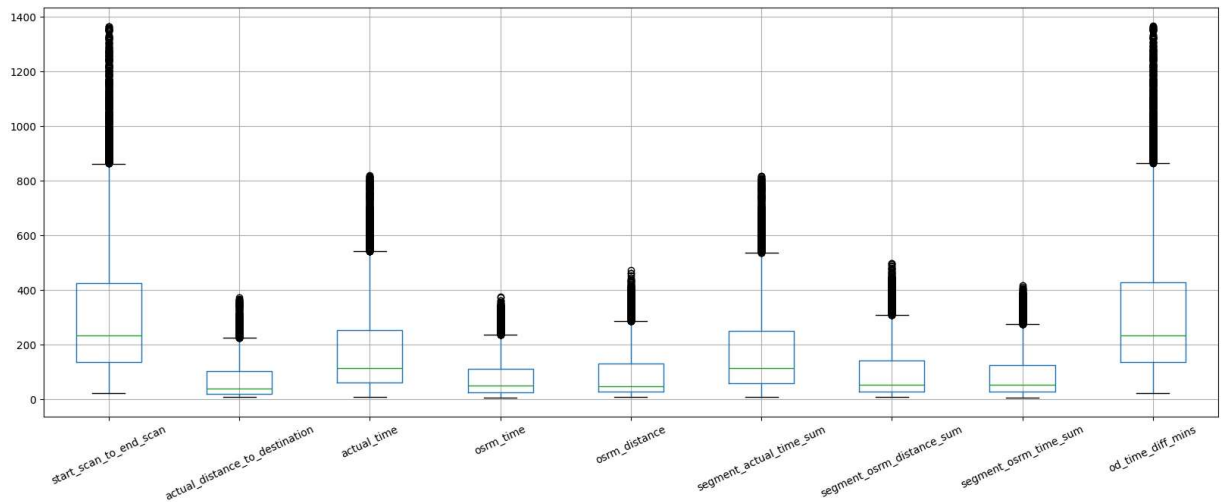

Out []:

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	st
1	training	2018-09-12 00:00:22.886430	thanos::sroute:3a1b0ab2- bb0b-4c53-8c59- eb2a2c0...	Carting	153671042288605164	IN
3	training	2018-09-12 00:01:00.113710	thanos::sroute:f0176492- a679-4597-8332- bbd1c7f...	Carting	153671046011330457	IN
4	training	2018-09-12 00:02:09.740725	thanos::sroute:d9f07b12- 65e0-4f3b-bec8- df06134...	FTL	153671052974046625	IN
5	training	2018-09-12 00:02:34.161600	thanos::sroute:9bf03170- d0a2-4a3f-aa4d- 9aaab3d...	Carting	153671055416136166	IN
6	training	2018-09-12 00:04:22.011653	thanos::sroute:a97698cc- 846e-41a7-916b- 88b1741...	Carting	153671066201138152	IN

5 rows × 32 columns

In []: `trip[numeric_cols].boxplot(rot=25, figsize=(20,7))`

Out []: <Axes: >



In []: `i=1
plt.figure(figsize=(25,10))
for col in numeric_cols:
 plt.subplot(3,3,i)
 sns.distplot(trip[col])
 i+=1`

```
/tmp/ipython-input-40587143.py:5: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])
```

```
/tmp/ipython-input-40587143.py:5: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])
```

```
/tmp/ipython-input-40587143.py:5: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])
```

```
/tmp/ipython-input-40587143.py:5: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])
```

```
/tmp/ipython-input-40587143.py:5: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])
```

```
/tmp/ipython-input-40587143.py:5: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])  
/tmp/ipython-input-40587143.py:5: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])  
/tmp/ipython-input-40587143.py:5: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

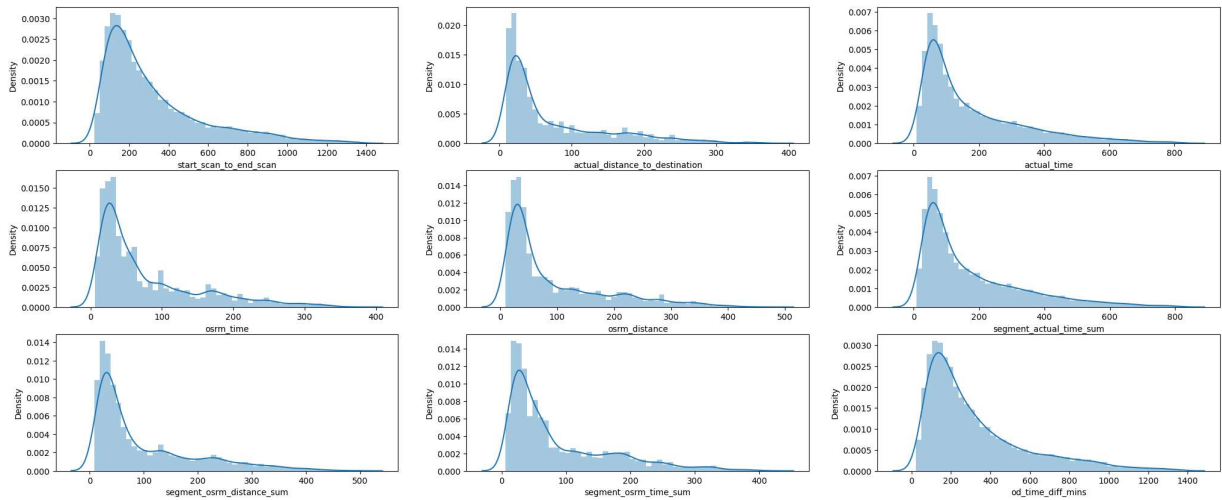
```
sns.distplot(trip[col])  
/tmp/ipython-input-40587143.py:5: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(trip[col])
```



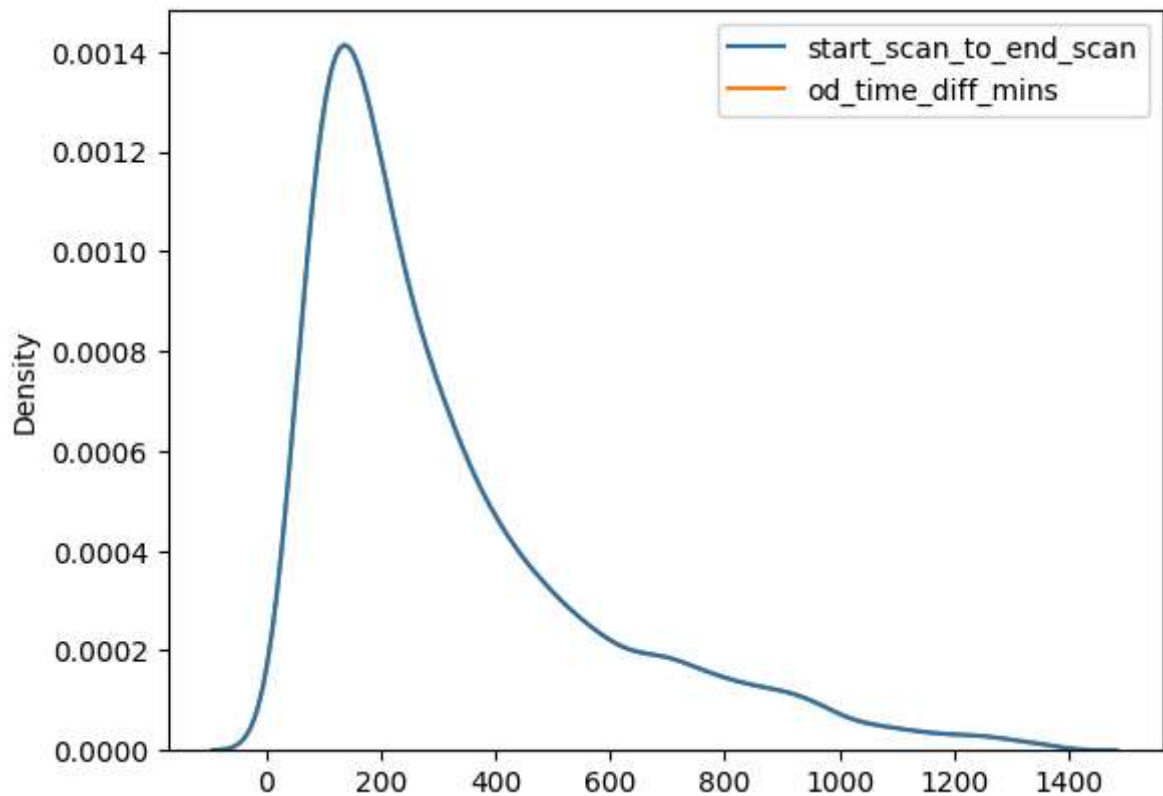
- All the numeric columns are right skewed as these attributes can't take negative values

Comparing the time and Distance fields

Visual Representation of start_scan_to_end_scan Vs od_time_diff_mins

```
In [ ]: sns.kdeplot([trip['start_scan_to_end_scan'], trip['od_time_diff_mins']])
```

```
Out[ ]: <Axes: ylabel='Density'>
```



Describing start_scan_to_end_scan Vs od_time_diff_mins

```
In [ ]: trip[['start_scan_to_end_scan', 'od_time_diff_mins']].describe()
```

```
Out [ ]:
```

	start_scan_to_end_scan	od_time_diff_mins
count	12744.000000	12744.000000
mean	322.166824	323.014710
std	257.550322	257.882213
min	23.000000	23.461468
25%	136.000000	136.810326
50%	234.000000	234.379203
75%	427.000000	428.355026
max	1366.000000	1367.376591

- Graphically and non-graphically we see very minor difference between start_scan_to_end_scan and od_time_diff_mins
- Now we need to check if the difference is statistically significant or not

Hypothesis testing for start_scan_to_end_scan and od_time_diff_mins : to validate if both have the same mean or not

- Ho : start_scan_to_end_scan and od_time_diff_mins have equal mean
- Ha : start_scan_to_end_scan and od_time_diff_mins have different mean
- Confidence Level : 95%
- alpha : 0.05

```
In [ ]: sstes = trip['start_scan_to_end_scan'].sample(5000)
otdm = trip['od_time_diff_mins'].sample(5000)
```

```
In [ ]: from scipy.stats import ttest_ind

test_stat, p_value = ttest_ind(ssstes,otdm)
```

```
print('Test Statistic : ', test_stat)
print('P-Value : ', p_value)
```

Test Statistic : 1.8960159359814084

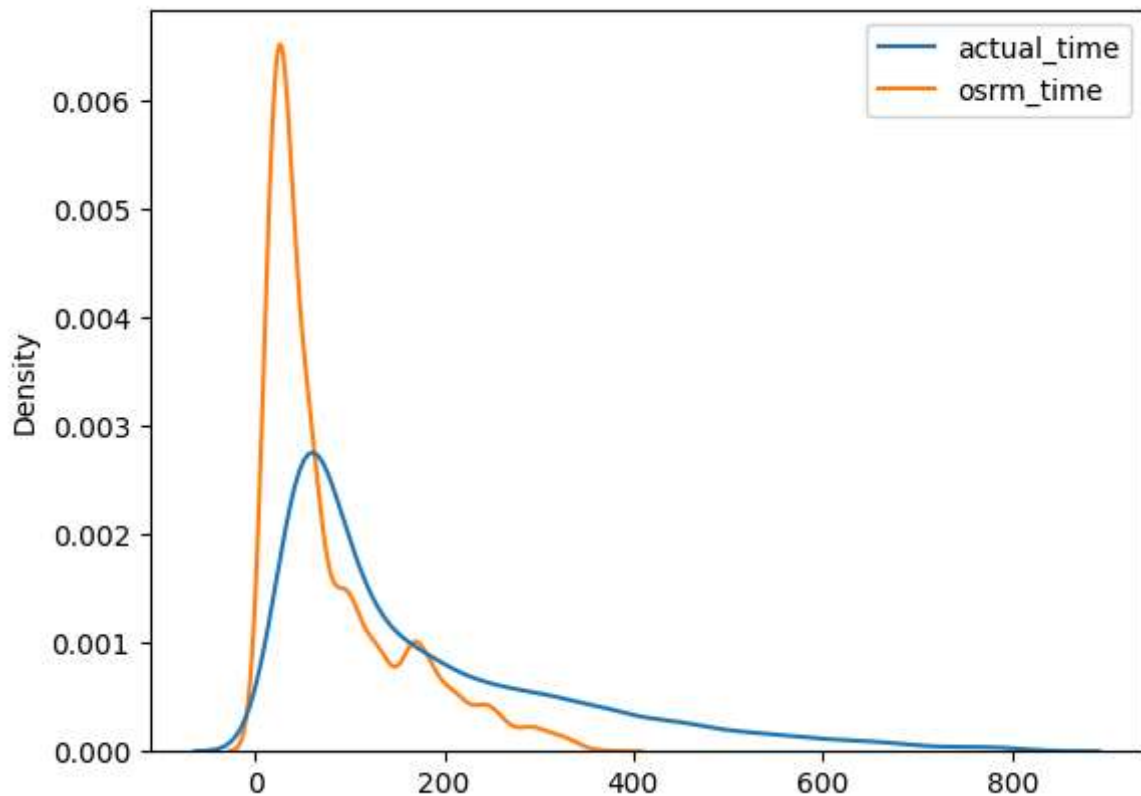
P-Value : 0.05798674287115718

- As P-Value is greater than alpha, we fail to reject H_0
 - H_0 : start_scan_to_end_scan and od_time_diff_mins have the equal mean
- We can conclude that there is no significant difference between start_scan_to_end_scan and od_time_diff_mins

Visual Representation of actual_time Vs osrm_time

```
In [ ]: sns.kdeplot([trip['actual_time'], trip['osrm_time']])
```

```
Out[ ]: <Axes: ylabel='Density'>
```



- Graphically and non-graphically we see actual_time Vs osrm_time follows different distribution
- Now we need to check if the difference is statistically significant or not

Describing actual_time Vs osrm_time


```
In [ ]: trip[['actual_time', 'osrm_time']].describe()
```

```
Out [ ]:
```

	actual_time	osrm_time
count	12744.000000	12744.000000
mean	178.612837	78.992546
std	159.187899	72.898179
min	9.000000	6.000000
25%	61.000000	27.000000
50%	115.000000	50.000000
75%	254.000000	111.000000
max	820.000000	376.000000

Hypothesis testing for actual_time and osrm_time : to validate if both have the same mean or not

- Ho : actual_time and osrm_time have equal mean
- Ha : actual_time and osrm_time have different mean
- Confidence Level : 95%
- alpha : 0.05

```
In [ ]: at = trip['actual_time'].sample(5000)
ot = trip['osrm_time'].sample(5000)
```

```
In [ ]: from scipy.stats import ttest_ind

test_stat, p_value = ttest_ind(at, ot)
print('Test Statistic : ', test_stat)
print('P-Value : ', p_value)
```

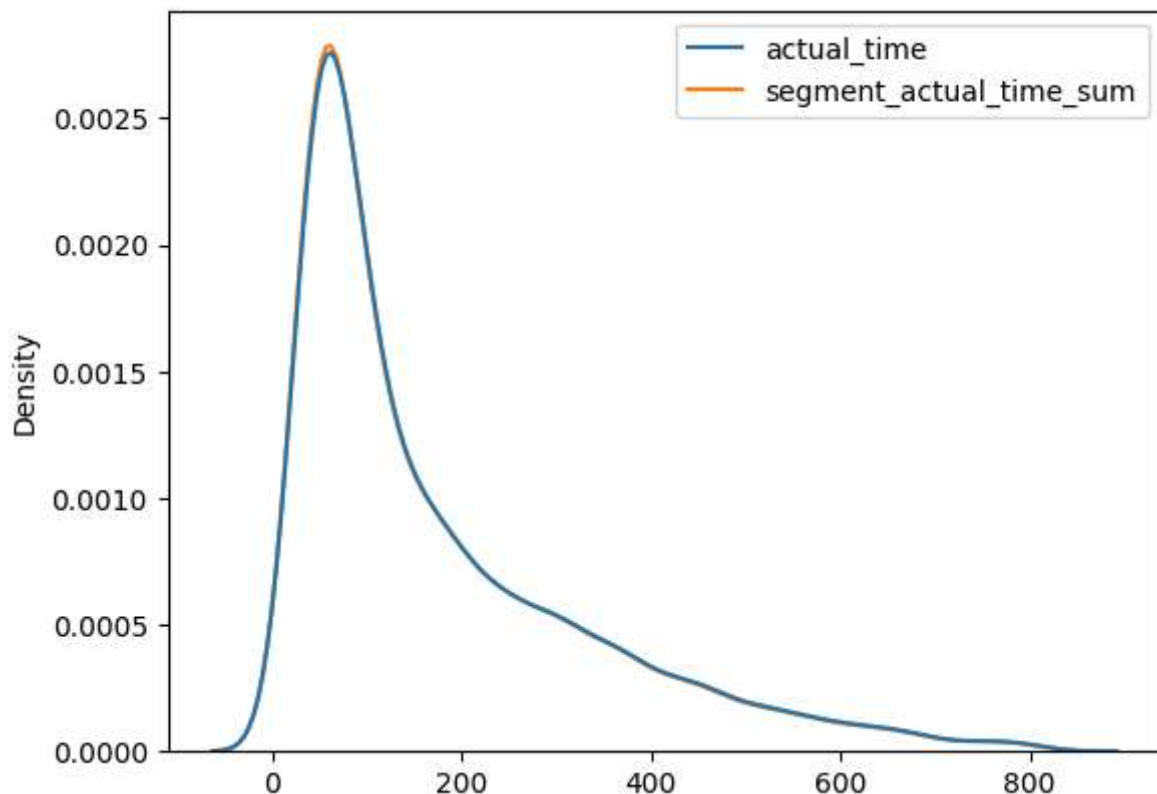
```
Test Statistic : 41.35299976609893
P-Value : 0.0
```

- As P-Value is smaller than alpha, we reject Ho
 - Ha : actual_time and osrm_time have different mean
- We can conclude that there is significant difference between actual_time and osrm_time

Visual Representation of actual_time Vs segment_actual_time_sum

```
In [ ]: sns.kdeplot([trip['actual_time'], trip['segment_actual_time_sum']])
```

```
Out [ ]: <Axes: ylabel='Density'>
```



Describing actual_time Vs segment_actual_time_sum

```
In [ ]: trip[['actual_time', 'segment_actual_time_sum']].describe()
```


Out[]:

	actual_time	segment_actual_time_sum
count	12744.000000	12744.000000
mean	178.612837	176.948682
std	159.187899	158.133558
min	9.000000	9.000000
25%	61.000000	60.000000
50%	115.000000	114.000000
75%	254.000000	251.000000
max	820.000000	818.000000

- Graphically and non-graphically we see very minor difference between actual_time and segment_actual_time_sum
- Now we need to check if the difference is statistically significant or not

Hypothesis testing for actual_time and segment_actual_time_sum : to validate if both have the same mean or not

- Ho : actual_time and segment_actual_time_sum have equal mean
- Ha : actual_time and segment_actual_time_sum have different mean
- Confidence Level : 95%
- alpha : 0.05

```
In [ ]: at = trip['actual_time'].sample(5000)
sats = trip['segment_actual_time_sum'].sample(5000)
```

```
In [ ]: from scipy.stats import ttest_ind

test_stat, p_value = ttest_ind(at, sats)
print('Test Statistic : ', test_stat)
print('P-Value : ', p_value)
```

Test Statistic : 1.0192472679893512
P-Value : 0.308110253927865

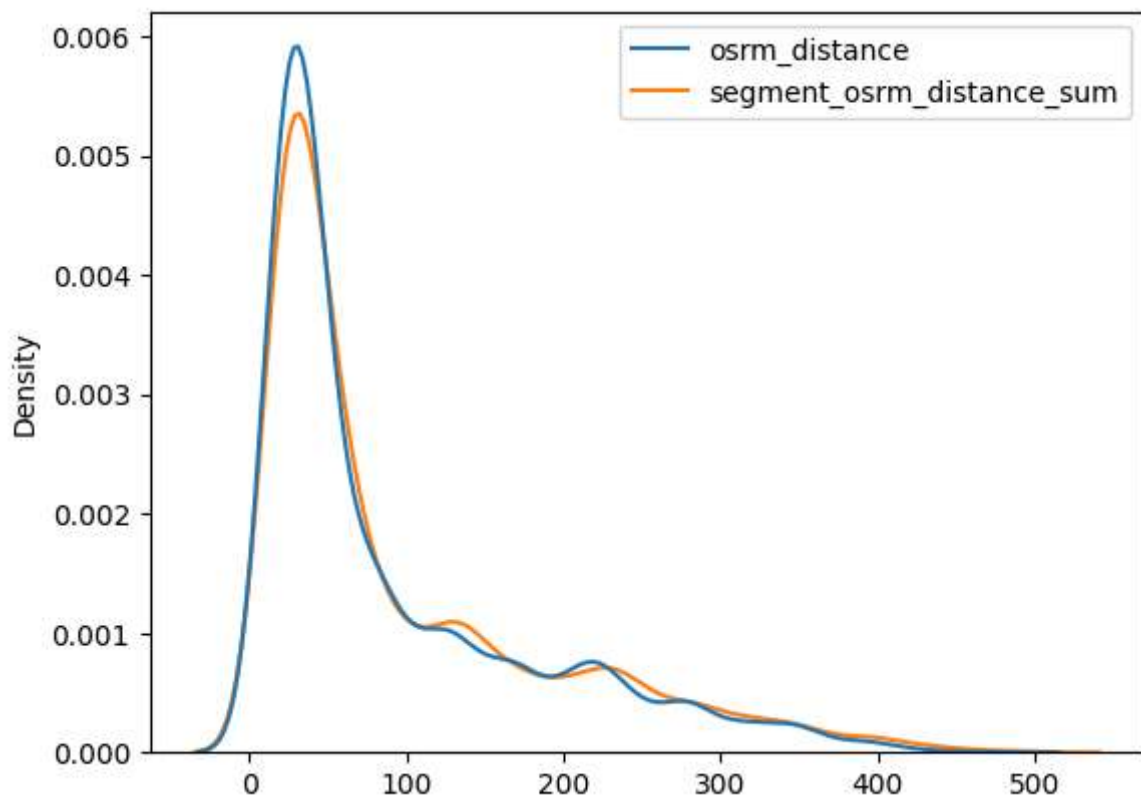
- As P-Value is greater than alpha, we fail to reject Ho

- H_0 : actual_time and segment_actual_time_sum have equal mean
- We can conclude that there is no significant difference between actual_time and segment_actual_time_sum

Visual Representation of osrm_distance Vs segment_osrm_distance_sum

```
In [ ]: sns.kdeplot([trip['osrm_distance'], trip['segment_osrm_distance_sum']])
```

```
Out[ ]: <Axes: ylabel='Density'>
```



Describing osrm_distance and segment_osrm_distance_sum

```
In [ ]: trip[['osrm_distance', 'segment_osrm_distance_sum']].describe()
```

Out[]:

	osrm_distance	segment_osrm_distance_sum
count	12744.000000	12744.000000
mean	92.396940	98.697578
std	90.244340	95.577469
min	9.072900	9.072900
25%	28.357800	29.464775
50%	48.619050	54.226200
75%	132.065825	141.580900
max	474.133700	497.792900

Hypothesis testing for osrm_distance and segment_osrm_distance_sum : to validate if both have the same mean or not

- Ho : osrm_distance and segment_osrm_distance_sum have equal mean
- Ha : osrm_distance and segment_osrm_distance_sum have different mean
- Confidence Level : 95%
- alpha : 0.05

```
In [ ]: od = trip['osrm_distance'].sample(5000)
sods = trip['segment_osrm_distance_sum'].sample(5000)
```

```
In [ ]: from scipy.stats import ttest_ind

test_stat, p_value = ttest_ind(od,sods)
print('Test Statistic : ', test_stat)
print('P-Value : ', p_value)
```

Test Statistic : -2.4629403047896328
P-Value : 0.013797014498887886

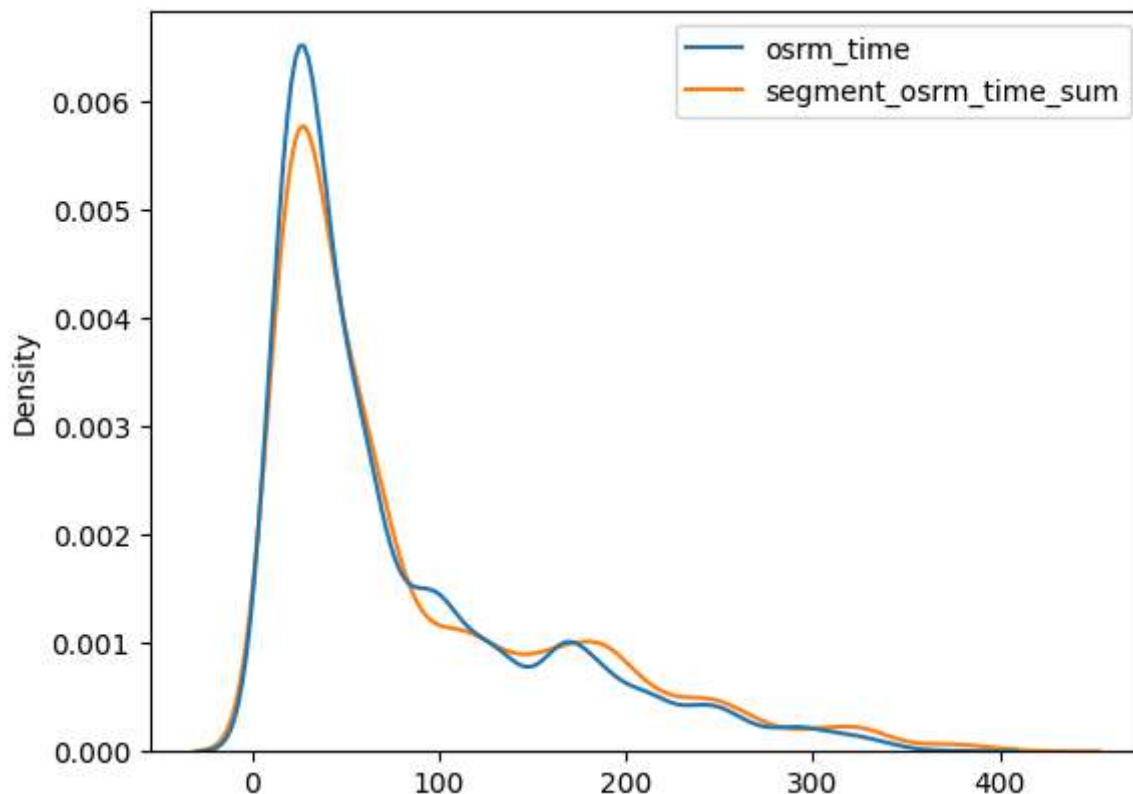
- As P-Value is greater than alpha, we fail to reject Ho
 - Ho : osrm_distance and segment_osrm_distance_sum have equal mean
- We can conclude that there is no significant difference between osrm_distance and segment_osrm_distance_sum

Add blockquote

Visual Representation of osrm_time and segment_osrm_time_sum

```
In [ ]: sns.kdeplot([trip['osrm_time'], trip['segment_osrm_time_sum']])
```

```
Out[ ]: <Axes: ylabel='Density'>
```



Describing osrm_time and segment_osrm_time_sum

```
In [ ]: trip[['osrm_time', 'segment_osrm_time_sum']].describe()
```

Out[]:

	osrm_time	segment_osrm_time_sum
count	12744.000000	12744.000000
mean	78.992546	86.527150
std	72.898179	80.244315
min	6.000000	6.000000
25%	27.000000	28.000000
50%	50.000000	53.000000
75%	111.000000	127.000000
max	376.000000	417.000000

Hypothesis testing for osrm_time and segment_osrm_time_sum : to validate if both have the same mean or not

- Ho : osrm_time and segment_osrm_time_sum have equal mean
- Ha : osrm_time and segment_osrm_time_sum have different mean
- Confidence Level : 95%
- alpha : 0.05

```
In [ ]: ot = trip['osrm_time'].sample(5000)
        sots = trip['segment_osrm_time_sum'].sample(5000)
```

```
In [ ]: from scipy.stats import ttest_ind

        test_stat, p_value = ttest_ind(ot,sots)
        print('Test Statistic : ', test_stat)
        print('P-Value : ', p_value)
```

Test Statistic : -5.4671267539897554
P-Value : 4.683901655635288e-08

- As P-Value is greater than alpha, we fail to reject Ho
 - Ho : osrm_time and segment_osrm_time_sum have equal mean
- We can conclude that there is no significant difference between osrm_time and segment_osrm_time_sum

Handling Categorical Variables

```
In [ ]: trip['route_type'].value_counts()
```

```
Out [ ]:          count
```

route_type	
Carting	8816
FTL	3928

dtype: int64

```
In [ ]: # One Hot Encoding
trip['route_type'] = trip['route_type'].map({'FTL':0, 'Carting':1})
```

```
In [ ]: trip['route_type'].value_counts()
```

```
Out [ ]:          count
```

route_type	
1	8816
0	3928

dtype: int64

Standardizing the Numercal Columns

```
In [ ]: from sklearn.preprocessing import StandardScaler
```

```
In [ ]: scaler = StandardScaler()
scaler.fit(trip[numeric_cols])
trip[numeric_cols] = scaler.transform(trip[numeric_cols])
```

```
In [ ]: trip[numeric_cols]
```

Out []:

	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm
1	-0.552018	0.004770	-0.223725	-0.150799	
3	-0.862649	-0.766652	-0.751424	-0.877869	
4	1.533093	0.752089	1.020137	0.521398	
5	-0.517072	-0.664436	-0.738859	-0.768123	
6	-0.870415	-0.877862	-0.971298	-0.905306	
...	...	...	...	...	...
14812	-0.253036	-0.207666	-0.600652	-0.233109	
14813	-1.017965	-0.789535	-0.990144	-0.919024	
14814	0.383758	-0.470411	0.649492	-0.425165	
14815	0.096424	0.852289	0.536413	1.371932	
14816	0.119722	-0.093089	0.605517	-0.150799	

12744 rows × 9 columns

In []:

```
trip[numeric_cols].describe()
```

Out []:

	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time
count	1.274400e+04	1.274400e+04	1.274400e+04	1.274400e+04
mean	3.289550e-17	1.003591e-17	-7.805711e-18	-9.478364e-17
std	1.000039e+00	1.000039e+00	1.000039e+00	1.000039e+00
min	-1.161631e+00	-8.792127e-01	-1.065530e+00	-1.001334e+00
25%	-7.228651e-01	-7.084152e-01	-7.388593e-01	-7.132494e-01
50%	-3.423420e-01	-4.707926e-01	-3.996242e-01	-3.977285e-01
75%	4.070556e-01	4.176759e-01	4.735920e-01	4.390879e-01
max	4.053088e+00	4.140033e+00	4.029278e+00	4.074438e+00

Insights

In []:

```
trip['source_state'].value_counts().sort_values(ascending=False).head(3)
```

Out[]: **count**

source_state	
maharashtra	2309
karnataka	2025
haryana	1376

dtype: int64

```
In [ ]: trip['source_state'].value_counts().sort_values(ascending=True).head(3)
```

Out[]: **count**

source_state	
arunachal pradesh	3
nagaland	4
himachal pradesh	12

dtype: int64

```
In [ ]: trip[trip['source_state']=='maharashtra']['source_city'].value_counts().head(5)
```

Out[]: **count**

source_city	
mumbai	666
bhiwandi	541
pune	357
mumbai hub	300
lowerparel	65

dtype: int64

```
In [ ]: trip['source_city'].value_counts().sort_values(ascending=False).head(3)
```


Out[]:

count	
source_city	
bengaluru	1113
gurgaon	753
mumbai	666

dtype: int64

```
In [ ]: trip['source_city'].value_counts().sort_values(ascending=True).head(3)
```

Out[]:

count	
source_city	
jetpur	1
kadiri	1
vadakkencherry	1

dtype: int64

```
In [ ]: trip.groupby(['source_state','destination_state'])['trip_uuid'].count().sort_values
```

Out[]: **trip_uuid**

source_state	destination_state	
maharashtra	maharashtra	2276
karnataka	karnataka	2007
tamil nadu	tamil nadu	1016
haryana	haryana	795
telangana	telangana	669
...	...	...
karnataka	goa	1
tamil nadu	andhra pradesh	1
	kerala	1
uttar pradesh	uttarakhand	1
west bengal	orissa	1

90 rows × 1 columns

dtype: int64

In []: `trip.groupby(['source_center','destination_center'])['trip_uuid'].count().sort_valu`

Out[]: **trip_uuid**

source_center	destination_center	
IND562132AAA	IND560300AAA	151
IND560099AAB	IND560300AAA	121
IND560300AAA	IND562132AAA	108
IND421302AAG	IND400072AAB	105
IND400072AAD	IND421302AAG	99
...	...	...
IND848114AAA	IND842001AAA	1
IND000000AAZ	IND444005AAB	1
	IND444203AAA	1
IND854326AAB	IND854105AAB	1
IND000000ACB	IND123106AAB	1

1991 rows × 1 columns

dtype: int64

In []: `np.mean(trip[(trip['source_center']=='IND562132AAA') & (trip['destination_center']`

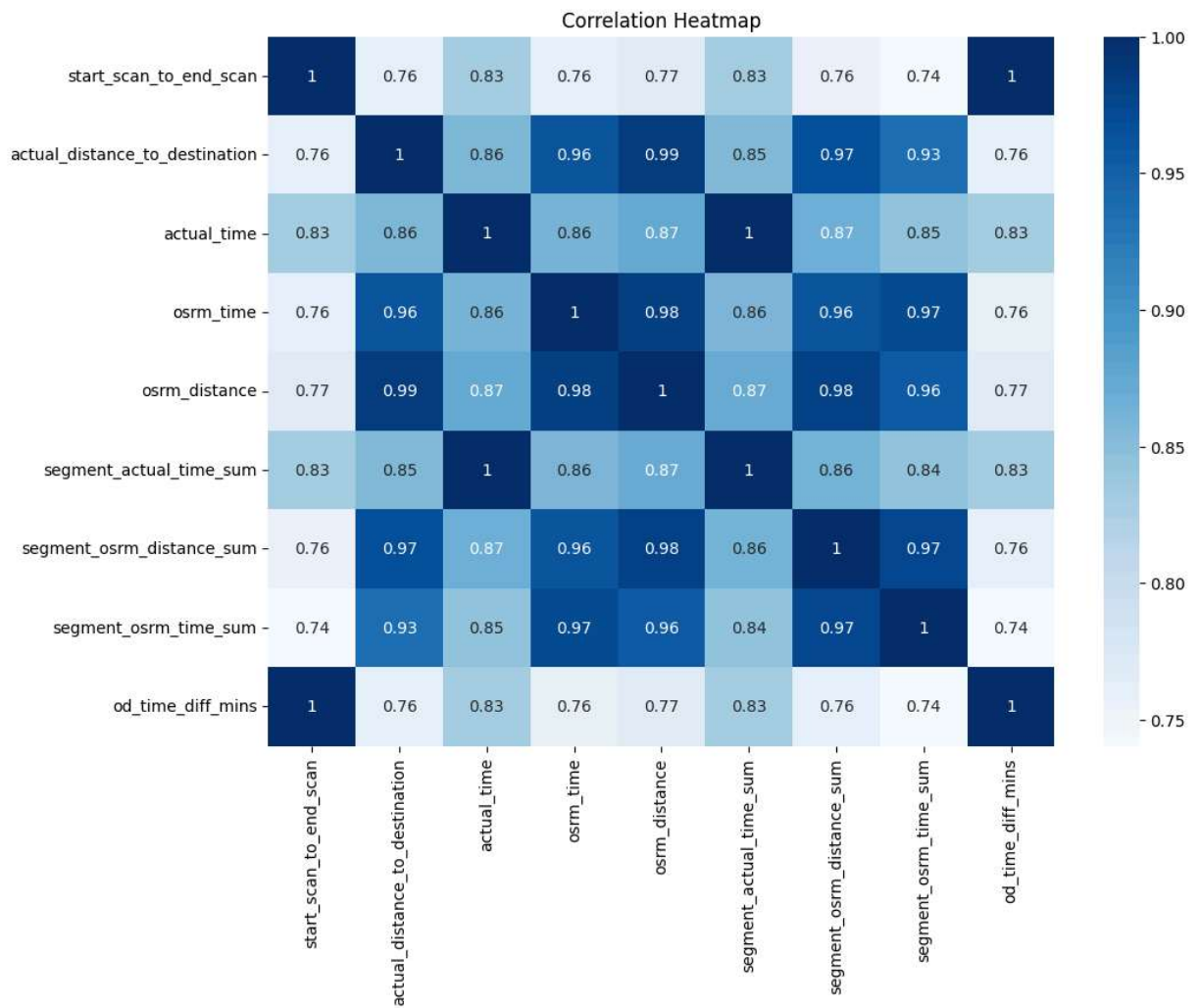
Out[]: `np.float64(-0.6172637504584045)`

In []: `np.mean(trip[(trip['source_center']=='IND562132AAA') & (trip['destination_center']`

Out[]: `np.float64(-0.5700321837180513)`

In []: `plt.figure(figsize=(11,8))
plt.title('Correlation Heatmap')
sns.heatmap(trip[numeric_cols].corr(method = 'pearson'),annot=True,cmap='Blues')`

Out[]: `<Axes: title={'center': 'Correlation Heatmap'}>`



```
In [ ]: trip[trip['source_state']==trip['destination_state']]['trip_uuid'].count()/len(trip
```

```
Out[ ]: np.float64(0.8666038920276209)
```

```
In [ ]: trip['trip_dayofweek'].value_counts()
```

Out[]:

	count
trip_dayofweek	
2	2357
5	1839
3	1823
4	1777
1	1769
0	1697
6	1482

dtype: int64

Business Insights

- Max orders are coming from state Maharashtra and the top 3 contributor cities are Mumbai, Bhiwandi, Pune
- Least orders were booked in Arunachal Pradesh, Nagaland, Himachal Pradesh
- At city level max orders are coming from Bengaluru, Gurgaon, Mumbai
- At city level min orders were placed Jetpur, Kadiri, Vadakkencherry
- Some cities appear under multiple names. Eg: Mumbai has been referred to as Mumbai, Mumbai hub, Lower Parel etc. If these different city names identify the same city, it could alter which city ranks highest in deliveries.
- Most of the orders are within the state (86.67%)
- Busiest route is IND562132AAA (bangalore_nelmngla_h (karnataka)) to IND560300AAA (bengaluru_kgairprt_hb (karnataka))
 - Average distance : 28.022 Km
 - Average time taken : 87.87 mins
- Max orders were booked on Wednesday followed by Saturday
- There is a significant difference between OSRM and actual parameters

Recommendation

There is a need to:

- Revisit information fed to routing engine for trip planning. Check for discrepancies with transporters, if the routing engine is configured for optimum results.

- North, South and West Zones corridors have significant traffic of orders. But, we have a smaller presence in Central, Eastern and North-Eastern zone. However it would be difficult to conclude this, by looking at just 2 months data. It is worth investigating and increasing our presence in these regions.
- More campaign / advertisement in the states with less orders like Arunachal, Nagaland, Himachal etc.
- From state point of view, we have heavy traffic in Maharashtra followed by Karnataka. This is a good indicator that we need to plan for resources on ground in these 2 states on priority. Especially, during festive seasons.
- Some cities appear under multiple names. Eg: Mumbai has been referred to as Mumbai, Mumbai hub, Lower Parel etc. If these different city names identify the same city, it could alter which city ranks highest in deliveries. Hence, it is recommended to clarify whether these names should be consolidated under one city or treated as separate locations

In []: